Making a pinball game for Playdate: Part 05, the spatial partition

Thursday, December 5, 2024

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-05-the-spatial-partition

Welcome to this December adventure, where I will write about the process of our latest game, Devils on the Moon pinball.

If I could only choose one book to recommend from this adventure, that would be Real-Time Collision Detection by Christer Ericson. Collision detection is a subject that has come up a bunch of times in my career, and it's something that once you learn can apply to a bunch of things in game dev.



Narrow collision detection

We already had all the primitive checks (minus one that I will cover in a later post) that we needed to make our game, the most complex one being the polygon collision test. The Pikuma 2D physics course covered the Separating axis theorem, and I had already understood and implemented that. But I had heard about the GJK algorithm for a while and wanted to try it out.

After watching a couple of videos and reading some articles, I understood some of it, but wasn't that confident in being able to implement it in a language I wasn't comfortable with yet. Not only that, but this was a basic piece we needed for our game. I remembered reading about the cute headers single header libraries and specifically knew there was a header for collision detection.

After reading the documentation, I got excited; not only did it supported polygons using GJK, but it also had a way to do swept collision detection, which at the time sounded useful.

Turns out we didn't need the swept functions, as running our physics step 4 times every frame and doing sequential impulses was enough to avoid any tunneling issues, at least with the speed the ball was moving in our game. But I'm getting out of topic.

I quickly added the header library to my project and started to testing it out. It worked great.

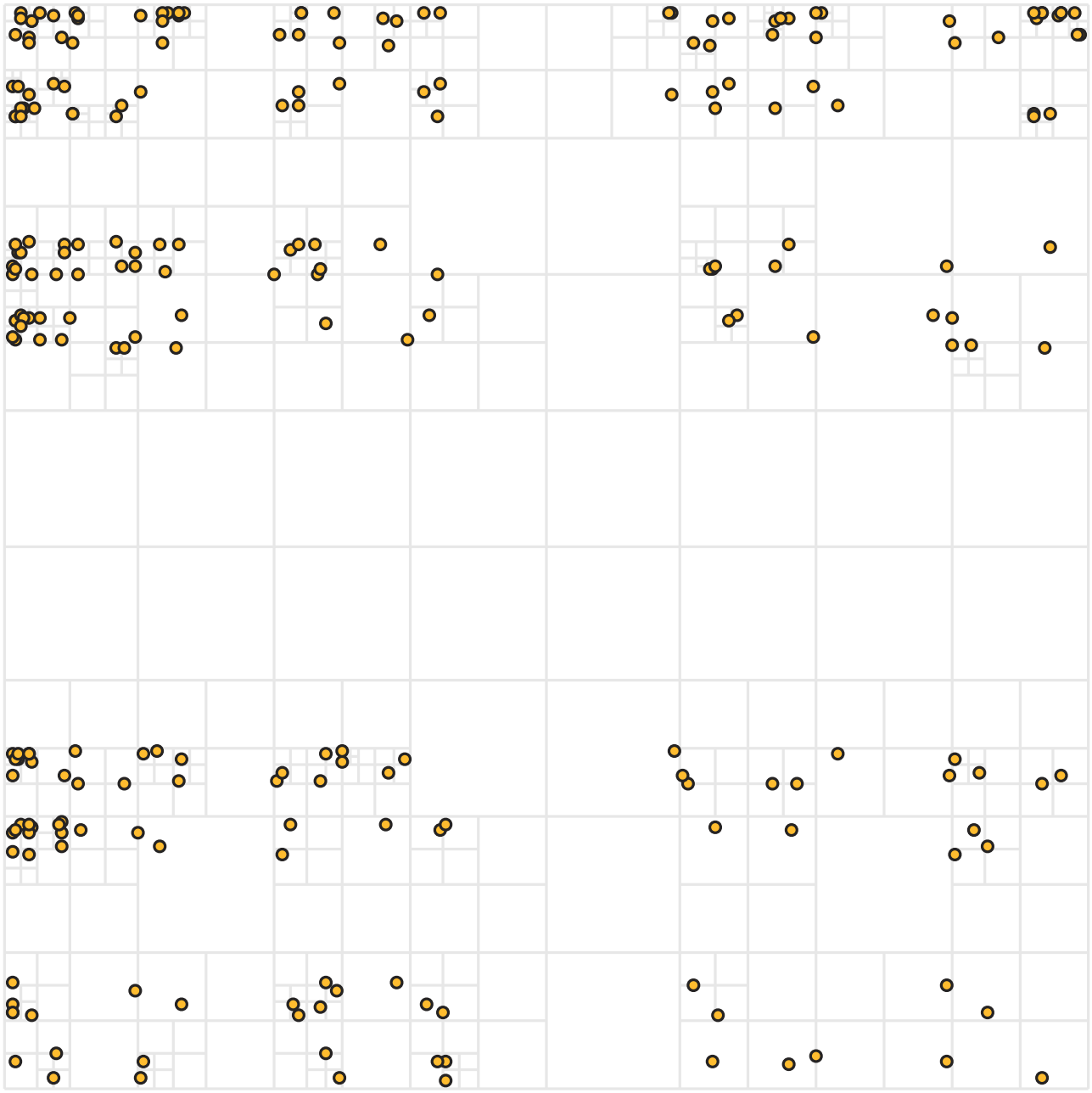
But as I mentioned before, after we added a bunch of polygons, the game started to slow down. I knew it was due to the lack of any spatial partitioning. I was checking every ball with every polygon every frame.

Broad collision detection

There are various ways to solve this, and Real-Time Collision Detection talks about a bunch of them, not only that but compares them and explains in which case you would use each one of the different techniques.

The main goal is to split the checks you have to do by some kind of spatial characteristic.

For example, Quad trees which I knew thanks to The Coding Train (Jp & I are fans). Quad trees are a technique where you subdivide the space into 4 parts. You put each element inside of one of these 4 parts, and if any part has more than a certain number of elements, you split that part into smaller 4 parts and repeat.



By David Eppstein

I always wanted to try to implement them; it didn't sound terrible complicated, and what better moment. But I decided to keep reading and see what the difference was with the other techniques.

> "A very effective space subdivision scheme is to overlay space with a regular grid. This grid divides spaces in to a number of regions, or grid cells of equal size. Each object is then associated with the cells it overlaps. Thanks to the uniformity of the grid, accessing a cell corresponding to a particular coordinate is both simple and fast." Real-Time Collision Detection by Christer Ericson

Well, I know grids. It's one of the most basic things you start doing while programming, and they are everywhere. Quad trees may sound more interesting, but nothing beats simple.

Not only that, but reading more, I realized our pinball table was mostly static, so storing all the static bodies in the grid could be done in a cache-friendly way.

> "When a grid data is static, the need for a linked list can be removed all together. Rather than storing the data in a linked list, the idea is to store all cell data in a single contiguous array." Real-Time Collision Detection by Christer Ericson

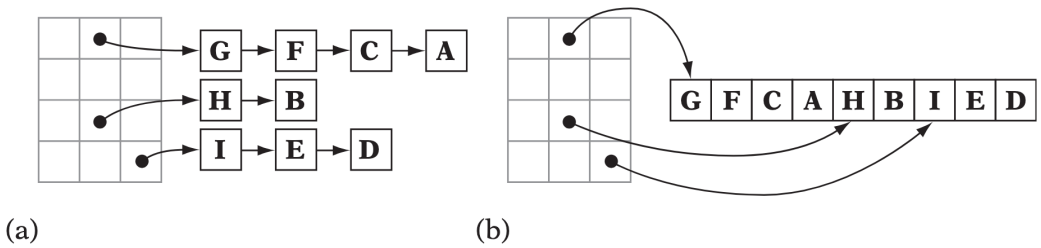


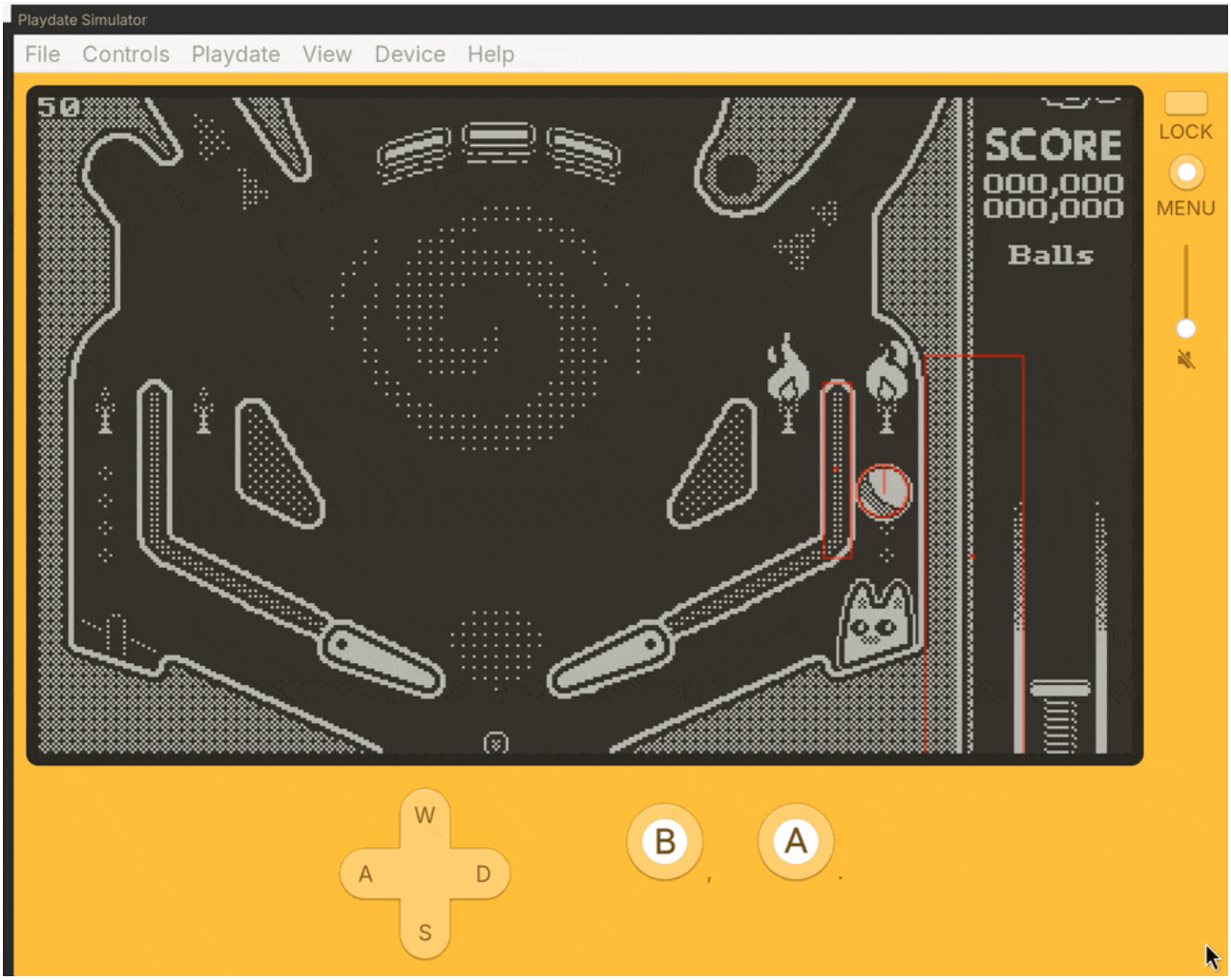
Figure 7.3 (a) A grid storing static data as lists. (b) The same grid with the static data stored into an array.

So I load all the polygon data from our Level Editor, order them by their top left corner, calculate the bounding box of each polygon, and store them in a contiguous array. I know the number of polygons beforehand thanks to the *level editor*, so I only need to allocate memory once.

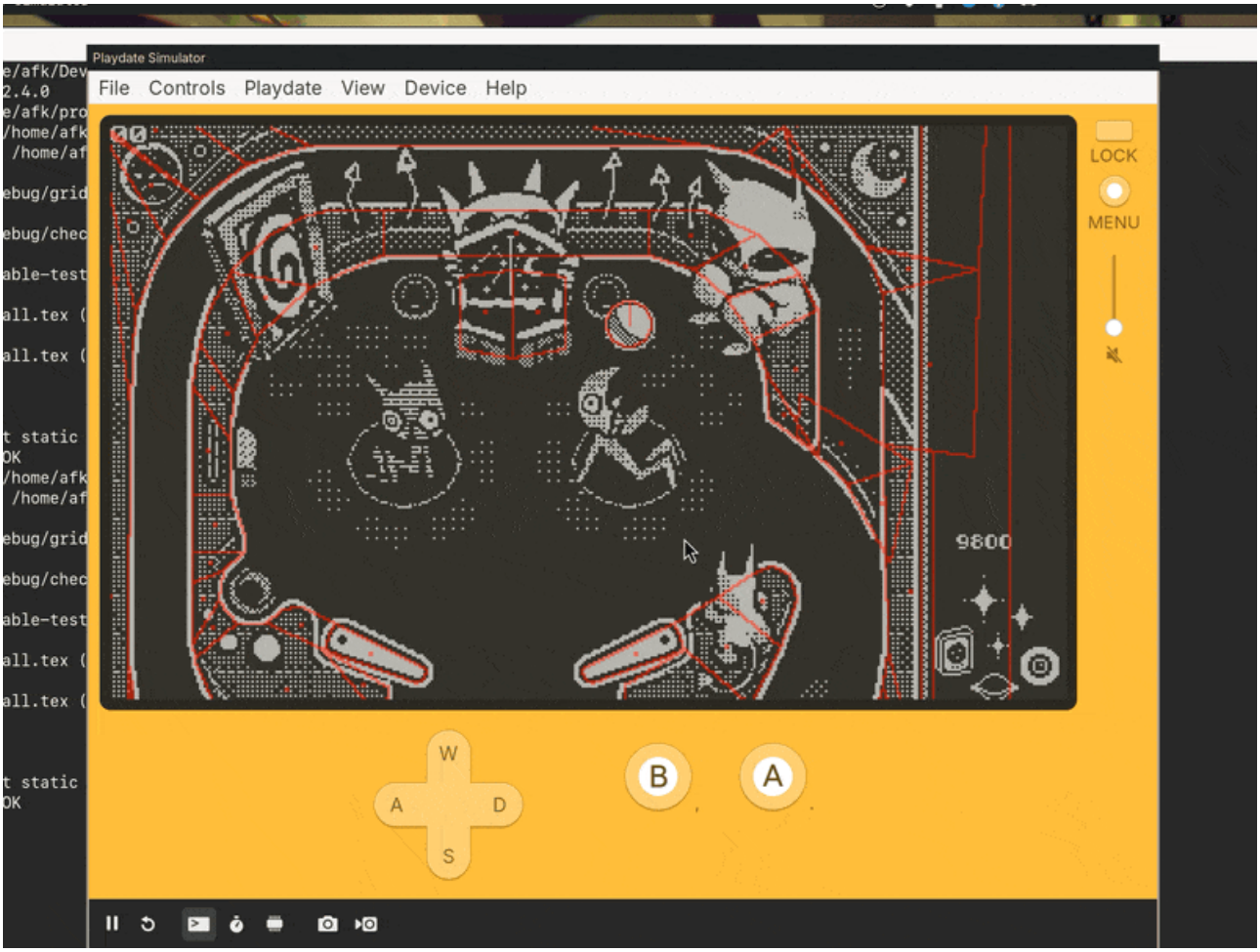
Then I calculate how many polygons each cell has, and store the polygons index on each cell they occupy, instead of storing them in a single cell and then having to check multiple ones when doing the broad face collision detection.

I have to run this logic twice, the first time to know how many elements each cell has, allocate the memory needed, and then run again to actually store the indices.

Not gonna lie, this process takes *some time*. I'm sure I'm doing it in the slowest way possible, but this only happens once when the game launches, I have it on my list of **TODO** to optimize it at some point, but if the game launches and the loading time is long, well you will know it was in exchange for really fast collision detection! (I mostly kid, I really want to fix it).



We filled the bottom part of the level with all the polygons we planned for the game and smooth 50fps, not a single frame drop.



Well, intuition and good practice can only take you so far. I wanted to be sure that if there were any performance issues, I had tools to diagnose them and fix them!

Will see you in the next adventure: The Profiler.